

RV32I Pipeline Branch Flush 구현 보고서

2026-06-25 | RV32I A* Accelerator SoC

프로젝트 개요

프로젝트명: RV32I A* Accelerator SoC

목표: RV32I 기반 CPU를 직접 구현하고, 이후 MMIO bus와 A* pathfinding accelerator를 통합하는 FPGA SoC 구현

타겟 보드: Terasic DE2-115 / Intel Cyclone IV

언어: Verilog HDL

현재 단계: 5-stage pipeline CPU의 branch 처리와 flush 검증

오늘 구현한 내용

오늘은 pipeline CPU에서 branch가 taken 되었을 때 잘못 들어온 instruction을 제거하는 flush 흐름을 구현하고 Questa로 검증했다.

핵심 변경은 branch 판단을 EX stage에서 수행하고, branch taken이면 IF/ID와 ID/EX pipeline register를 NOP 또는 bubble로 만드는 구조이다.

핵심 코드

```
wire branch_taken;  
assign branch_taken = E_branch && E_branch_condition;
```

설계 결정

Branch condition은 EX stage의 ALU 쪽에서 계산한다. ID stage 판단은 penalty를 줄일 수 있지만 forwarding/hazard 처리가 더 복잡해져 현재 단계에서는 EX stage 판단이 더 적절하다.

Branch가 EX stage에서 taken으로 판정되면 wrong-path instruction은 IF/ID와 ID/EX에 존재한다. 따라서 flush 대상은 IF/ID와 ID/EX이다.

IF/ID는 instruction을 NOP으로 바꾸고, ID/EX는 control signal을 모두 0으로 만들어 부작용 없는 bubble로 처리한다.

추가/검증 파일

File	Role
Risc_V/6_24_25/program.hex	Branch taken/not-taken verification instruction image
Risc_V/6_24_25/tb_branch_nohazard.v	Questa testbench checking register results
Risc_V/6_24_25/IF_ID_reg.v	Flush inserts NOP instruction
Risc_V/6_24_25/ID_EX_reg.v	Flush clears control signals to create a bubble

Questa 검증 결과

BEQ taken, BNE not taken, BLT taken, BGE taken을 포함한 program.hex와 tb_branch_nohazard.v를 추가했다.
아직 forwarding/stall이 없으므로 RAW hazard를 피하기 위해 dependent instruction 사이에 NOP을 삽입했다.
최종 결과: PASS: branch taken/not-taken flush test without RAW hazards

현재 한계

Forwarding unit, stall/hazard detection, load-use hazard 처리는 아직 구현 전이다.
JAL/JALR datapath는 아직 완성 전이며, MMIO bus와 A* accelerator도 통합 전이다.
따라서 현재 테스트는 hazard를 의도적으로 피한 instruction sequence로 구성했다.

다음 작업 계획

LW/SW를 포함한 data memory 테스트 작성
forwarding unit 설계
load-use stall 처리 추가
JAL/JALR PC update와 writeback 처리 정리
MMIO bus 설계 전 CPU core 인터페이스 정리

Questa 실행 명령

```
cd D:/Programs/vscode_workspace/Soc_Project/Risc_V/6_24_25
vlib work
vlog -sv *.v
vsim tb_branch_nohazard
run -all
```

포트폴리오 관점 의미

오늘 작업은 단순히 branch instruction을 하나 추가한 것이 아니라, pipeline CPU에서 control hazard가 왜 발생하고 어떻게 bubble/NOP으로 처리되는지를 직접 구현하고 검증한 단계이다. Questa에서 branch taken/not-taken testbench를 통과시켰기 때문에, 이후 forwarding과 hazard detection을 붙일 수 있는 pipeline 구조의 기반이 만들어졌다.